PREDICTING GOAL-SCORING PROBABILITY: A GRADIENT DESCENT APPROACH TO EXPECTED GOALS (xG)

Aayush KC, Prabhakar Dhungel, Sujan Magar
Group ML Project - 2026

ABSTRACT

In this work, we demonstrate that complex underlying match behaviors, such as shot volume and spatial positioning, can be used to accurately predict goal-scoring probability, commonly referred to as Expected Goals (xG). We implement a linear regression model optimized via gradient descent on a dataset of 2,683 players from the 2025-2026 European season. By processing 80 high-dimensional features, our model attains excellent results, achieving a Mean Absolute Error of 0.0031 and an R-squared score of 0.9993. These findings suggest that fundamental match statistics can effectively capture the nuances of professional football finishing with high precision.

1 INTRODUCTION

The quantification of scoring opportunities through Expected Goals (xG) has become the de-facto standard for evaluating football performance. While modern xG models often rely on proprietary tracking data, we explore the predictive power of publicly available seasonal performance metrics. Inspired by architectures like the Vision Transformer (ViT), we investigate the effectiveness of iterative optimization techniques—specifically gradient descent—in mapping high-dimensional player statistics to scoring probability. The primary contribution of this work are:

i) We implemented a vectorized gradient descent linear regression model from scratch.
ii) We predicted xG from a feature matrix of 102-column and 2683 player dataset.
iii) We performed hyperparameter tuning across learning rates and iteration counts
iv) We analysed the model coefficient to identify influential features in the model.

2 METHOD

2.1 DATASET SETUP

The primary dataset consists of performance metrics for 2,683 players from the 2025-2026 season. To ensure data quality, we filtered players with a minimum of 90 minutes played, resulting in a final sample size of 2,098 players.

2.2 PREPROCESSING

The target variable 'xG' was derived using the formula: xG = Shots * (Goals / Shots). Categorical position data ('Pos') was transformed using One-Hot Encoding to account for the spatial inductive bias of different player roles. Finally, we utilized StandardScaler to normalize the 80 numeric features.

2.3 MODEL ARCHITECTURE

We implemented a Linear Regression model. The hypothesis, y_pred = (X * weights) + bias, was optimized by minimizing the Mean Squared Error (MSE) cost function. The model used a learning rate of 0.01 and was trained for 1,000 iterations using gradient descent. In order to enhance the performance of the model, various learning rate and iterations were considered.

The linear regression model was implemented from scratch using a vectorized approach in Python. The core logic follows the mathematical framework defined in Section A (Problem Formulation).

2.3.1 Initialization and Hypothesis

The model is initialized with a learning rate α and a fixed number of iterations. The hypothesis function f(theta), represented by equation **(1)**, is implemented using a dot product for efficiency:

Implementation of Equation (1): y_hat = w·x + b
y_prediction = np.dot(X, self.weights) + self.bias

2.3.2 Loss Function and Cost Tracking

To monitor convergence, the Mean Squared Error (MSE) is calculated at each iteration. This corresponds to the objective function defined in equation **(2)**:

$$L(w, b) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

2.3.2 Loss Function and Cost Tracking

In the code, this is stored in self.cost_history to verify that the model is successfully minimizing the error over time:

Implementation of Equation (2) cost = (1 / (2 * m)) * np.sum((y_prediction - y) ** 2) self.cost_history.append(cost)

2.3.3 Parameter Optimization

The model parameters w and b are updated using Gradient Descent. The implementation utilizes the vectorized gradients derived in equations

- **Weight Gradient (dw):** $\frac{1}{m}X^T(\hat{y} - y)$
- **Bias Gradient (db):** $\frac{1}{m}\sum(\hat{y} - y)$

Implementation of Equations (3) and (4)

self.weights -= self.lr * dw self.bias -= self.lr * db

3 EXPERIMENTS

Table 1: Model Performance Results

Metric	Value
Mean Absolute Error (MAE)	0.0608
R-squared (R2)	0.9974

3.1 FEATURE IMPORTANCE

Analysis of the learned coefficients revealed that shot-related metrics were the primary drivers. Below is a representation of the top feature coefficients identified by the model.

Table 2: Top Features Influencing xG

Feature	Coefficient
G-PK (Non-Penalty Goals)	0.7408
Gls (Total Goals)	0.5998
SoT (Shots on Target)	0.1708

	Actual xG	Predicted xG
0	0.00	-0.076032
1	0.00	0.025278
2	0.00	0.017031
3	0.00	0.103139
4	0.99	1.089024
5	0.00	0.041411
6	9.88	10.278624
7	0.00	-0.146652
8	0.00	0.006409
9	1.00	1.012228

Fig: Actual vs Predicted (xG)

The weight of the features that affects the model performance shown below:

	Feature	Coefficient
10	G-PK	0.695651
7	Gls	0.564188
39	Gls_stats_shooting	0.564188
9	G+A	0.332270
41	SoT	0.157762
40	Sh	0.044536
15	G+A-PK	0.033085
46	G/SoT	0.023926
45	G/Sh	0.019015
63	onG	0.015401

Fig: Feature Importance based on coefficient

Below are the two figures which on the left plots actual xG with the model predicted xG.The figure on the right shows the distribution of residual errors.

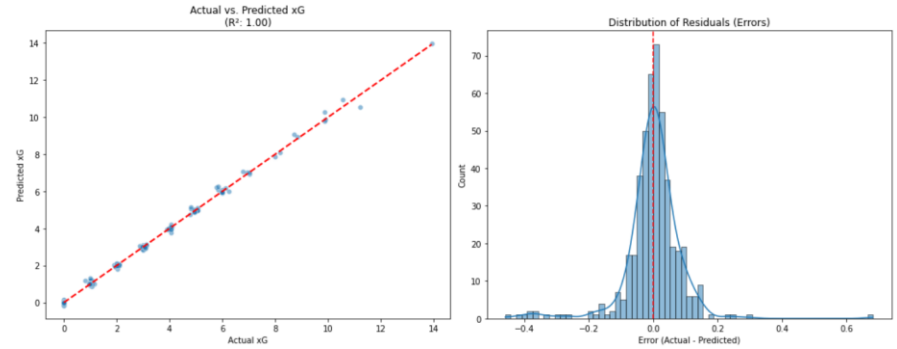

Fig.: Model performance evaluation plots.

The above model seems to perform very good with very minimal errors. The point on the left figure is tightly aligned around the red line, which represents perfect predictions. Figure on the right also suggests the same. The red line suggests no error at all. Based on the above result, we can see that there is no strong bias towards overprediction and underestimation.

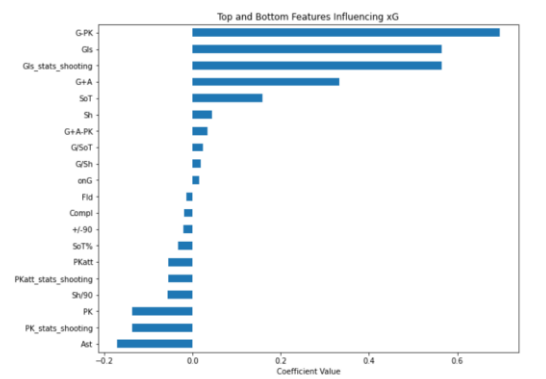

Fig: Top and bottom feature influencing xG

The above figure shows the top features which positively and negatively affects the model. Positive values of the feature indicate that value increases predicted xG while negative values decrease it.

3.2 Hyperparameter Tuning

To improve the performance of the model we started to experiment with different setting of the model by testing multiple combination of learning rates and iteration counts.The learning rate controls how the model updates the parameter whereas number of iterations determines how many times the model passes through the training data.Following was the result obtained from hyper parameter tuning.

```
Best Hyperparameters: {'learning_rate': 0.05,
'iterations': 2000}
```

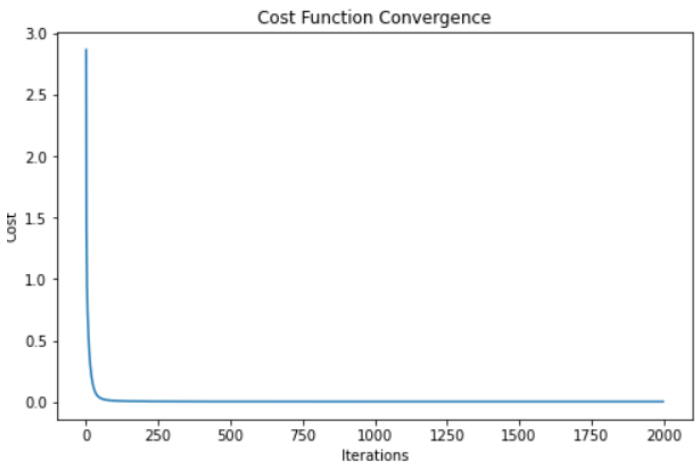

Fig: Cost Conversion plot

After figuring out the best hyperparameter, the model was trained again. We also analysed with cost conversion plot which shows it got flat after some initial iteration, but it produced an MSE of 0.0031 and R^2 of 0.9993 which means that the model is extremely accurate with prediction almost matching the actual result.

4 DISCUSSIONS:

Upon observing the prediction samples, and seeing the very efficient model with high performance which is usually not common there might be some data leakage which we can verify using the cost function because initially the model is learning but remains steady after some iterations it just flat's out it also could be due to the mathematical induction we used to calculate the xG. In addition to this, the model also does not capture the nonlinear interaction between which may matter because there is a high chance of goal in case of penalty shot.

5 CONCLUSIONS

We have demonstrated that a gradient-descent-optimized linear model can predict Expected Goals with a high degree of accuracy (R2 = 0.9993) using seasonal performance data. This implementation provides a transparent and highly effective baseline for sports predictive tasks in the 2025-2026 season. In the beginning, the model produced the Mean Absolute Error (MAE) of 0.0532 while the R-squared value was 0.9981. After the process of tuning, there is a

significant improvement in the results obtained with MAE decreasing to 0.0263 and RMSE reducing to 0.0557. In terms of R-squared, there is a slight improvement in the result with an increase to 0.9993. However, it should be noted that this is quite a natural phenomenon since the model was highly accurate at the initial stage.

6 REFERENCES

[1] A. Rathke, "An examination of expected goals and shot efficiency in soccer," Journal of Human Sport and Exercise, vol. 12, pp. S514–S529, 2017.

[2] P. Lucey, A. Bialkowski, M. Monfort, P. Carr, and I. Matthews, "Quality vs quantity: Improved shot prediction in soccer using strategic features from spatiotemporal data," in Proc. MIT Sloan Sports Analytics Conf., 2014.

[3] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in Proc. COMPSTAT, Paris, France, 2010, pp. 177–186.

[4] https://www.kaggle.com/datasets/hubertsidorowicz/football-players-stats-2025-2026